Part 10 — Cheat Sheets, Revision & Mock Interviews

One-Page Cheat Sheets

1. Azure DevOps Cheat Sheet

```
AZURE DEVOPS = Repos + Pipelines + Boards + Artifacts + Test Plans
Pipeline Structure: trigger → pool → variables → stages → jobs → steps
Agents: Microsoft-hosted (managed, fresh VM) | Self-hosted (your infra, private acc
YAML Keywords: trigger, pool, variables, stages, jobs, steps, template, extends, pa
Service Connection: Auth to external services (Azure, K8s, Docker). Use Workload Id
Variable Groups: Env-specific config. Link to Key Vault for secrets
Environments: Deployment targets with approval gates, checks, and history
Templates: Reusable YAML. extends (enforce), template (include)
Classic vs YAML: Always YAML for new projects (version-controlled, reviewable)
Approvals: Configured on Environments, not pipelines
Key command: az pipelines run --name "MyPipeline" --branch main
```

2. Terraform Cheat Sheet

```
COMMANDS: init → plan → apply → destroy | fmt | validate | import | state
STATE: Remote backend (S3/Azure Blob) + locking (DynamoDB/blob lease)
VARIABLES: string, number, bool, list, map, object | sensitive = true
MODULES: Reusable code. Pin with ?ref=v1.0.0. Test with Terratest
COUNT vs FOR_EACH: for_each for distinct resources (no index shift problem)
LIFECYCLE: create_before_destroy, prevent_destroy, ignore_changes
DATA SOURCES: Read existing resources. data "azurerm_subnet" "existing" {}
WORKSPACES: Separate state per workspace. Better: separate directories
STATE MGMT: state list, state show, state mv, state rm, state pull
IMPORT: terraform import azurerm_resource_group.main /subscriptions/.../rg
DRIFT: terraform plan detects. Apply to reconcile or update code
SENSITIVE: Mark variables/outputs sensitive. Never in .tfvars in Git
BACKEND: azurerm { storage_account, container, key } | s3 { bucket, key, dynamodb }
```

3. Kubernetes Cheat Sheet

```
ARCHITECTURE: API Server → etcd, Scheduler, Controller Manager | kubelet, kube-prox
WORKLOADS: Pod < ReplicaSet < Deployment | StatefulSet | DaemonSet | Job | CronJob
SERVICES: ClusterIP (internal) | NodePort (30000-32767) | LoadBalancer | Ingress
STORAGE: PV → PVC → Pod. StorageClass for dynamic provisioning
CONFIG: ConfigMap (non-sensitive) | Secret (base64, NOT encrypted by default)
SCALING: HPA (pods) + Cluster Autoscaler (nodes). Requires metrics-server
PROBES: liveness (restart if fails) | readiness (remove from service) | startup (sl
STRATEGY: RollingUpdate (default) | Recreate. maxUnavailable + maxSurge
SECURITY: RBAC (Role/ClusterRole + Binding) | NetworkPolicy | PodSecurity | Service
DEBUGGING: kubectl describe → kubectl logs → kubectl exec → kubectl get events
```

```
ROLLBACK: kubectl rollout undo deployment/<name>
NAMESPACES: Isolation boundary. ResourceQuota + LimitRange per namespace
KEY COMMANDS:
  kubectl get pods -A                    # All namespaces
  kubectl describe pod <name>            # Detailed info
  kubectl logs <pod> -c <container> -f   # Follow logs
  kubectl exec -it <pod> -- /bin/sh      # Shell into pod
  kubectl top pods                       # Resource usage
  kubectl get events --sort-by='.lastTimestamp'  # Recent events
```

4. Linux Cheat Sheet

```
FILES: find, ls, cat, head, tail, wc, du, df, ln, chmod, chown
PROCESS: ps aux, top, htop, kill, kill -9, nice, nohup, strace, lsof
NETWORK: ip addr, ss -tulnp, netstat, curl, wget, dig, nslookup, traceroute, nc
SYSTEMD: systemctl start/stop/enable/status, journalctl -u <service> -f
DISK: df -h, du -sh, lsblk, mount, fdisk, mkfs, lvextend
TEXT: grep -rni, sed 's/old/new/g', awk '{print $1}', cut -d: -f1, sort | uniq -c
SSH: ssh-keygen, ssh-copy-id, ssh -L (tunnel), scp, rsync
CRON: crontab -e, format: MIN HOUR DOM MON DOW cmd
PERMISSIONS: chmod 755, chown user:group, umask 022, SUID/SGID/Sticky
LOGS: /var/log/syslog, /var/log/auth.log, journalctl, dmesg
TROUBLESHOOTING:
  High CPU:    top → identify PID → strace -p PID
  Disk Full:   du -sh /* → find large files → rotate logs
  Can't SSH:   ping → nc -zv host 22 → check sshd → check firewall
  OOM:         free -h → ps aux --sort=-%mem → kill/optimize
```

5. Docker Cheat Sheet

```
BUILD: docker build -t name:tag . | Multi-stage: FROM ... AS builder
RUN:   docker run -d -p 8080:80 --name app image:tag
EXEC:  docker exec -it container /bin/sh
LOGS:  docker logs -f container
COMPOSE: docker compose up -d | down | logs -f | ps
DOCKERFILE: FROM → WORKDIR → COPY deps → RUN install → COPY . → EXPOSE → CMD
ENTRYPOINT vs CMD: ENTRYPOINT=executable(fixed), CMD=args(overridable)
VOLUMES: Named(-v pgdata:/data) | Bind(-v ./src:/app) | tmpfs
NETWORK: bridge(default) | host | none | overlay(multi-host)
OPTIMIZE: alpine base, multi-stage, .dockerignore, combine RUN, pin versions
SECURITY: non-root USER, scan with grype/trivy, no secrets in image
CLEANUP: docker system prune -a --volumes
```

6. Git Cheat Sheet

```
BASICS: clone, add, commit, push, pull, fetch, status, log, diff
BRANCH: branch, checkout -b, switch, merge, rebase, cherry-pick
UNDO: reset --soft/--hard HEAD~1, revert (new commit), reflog (recover)
STASH: stash, stash pop, stash list, stash drop
STRATEGY: Trunk-based (recommended) | GitFlow (complex teams)
MERGE vs REBASE: merge=preserve history, rebase=linear history
CONFLICT: Edit files → remove markers → git add → git commit
HOOKS: pre-commit(lint), pre-push(test), commit-msg(format)
```

```
SQUASH: rebase -i HEAD~N → squash
BISECT: bisect start → good → bad (binary search for bug)
```

7-11: Azure, Monitoring, Security, Networking, CI/CD

```
AZURE: Resource Groups → VNet/Subnet/NSG → VMs/AKS/App Service
       Key Vault (secrets) | ACR (images) | Azure Monitor (metrics)
       Managed Identity > Service Principal > Access Keys

MONITORING: Prometheus(metrics) + Grafana(dashboards) + Loki(logs)
            AlertManager → Slack/PagerDuty. PromQL for queries
            SLI(metric) → SLO(target) → SLA(contract)
            rate(), histogram_quantile(), sum by()

SECURITY: Shift-left: SAST→Build→Scan→DAST→Deploy
          RBAC everywhere (K8s, Azure, CI/CD)
          Secrets: Key Vault > K8s Secrets > Env vars. Never in code
          Network: Defense in depth (NSG + NetworkPolicy + App auth)

NETWORKING: OSI model, TCP vs UDP, DNS, HTTP/HTTPS
            VNet peering, VPN Gateway, Private endpoints
            Load Balancer (L4) vs Application Gateway (L7)
            K8s: ClusterIP→NodePort→LoadBalancer→Ingress

CI/CD: Commit→Build→Test→Scan→Stage→Prod
       Pipeline-as-code (Jenkinsfile/YAML). Artifact immutability
       Environment promotion with approvals. Rollback strategy
       DORA metrics: Lead Time, Frequency, MTTR, Change Failure Rate
```

Last Night Revision

30-Minute Revision (Emergency)

1. **Tell me about yourself** — rehearse 90-second version (2 min)
2. **Project architecture** — draw diagram from memory (3 min)
3. **Top 5 Docker questions** — ENTRYPOINT vs CMD, multi-stage, layers (3 min)
4. **Top 5 K8s questions** — Deployment vs StatefulSet, Services, CrashLoopBackOff debug (3 min)
5. **Top 5 Terraform questions** — state locking, count vs for_each, modules (3 min)
6. **Azure DevOps mapping** — Repos=GitLab, Pipelines=Jenkins, Artifacts=Nexus (3 min)
7. **Linux troubleshooting** — high CPU, disk full, can't SSH (3 min)
8. **Incident handling** — STAR example ready (3 min)
9. **Why Infosys + salary expectation** — rehearse (2 min)
10. **Review cheat sheets above** (5 min)

1-Hour Revision

- Everything in 30-min + add:
- CI/CD pipeline design — draw end-to-end flow (5 min)

- Git branching strategy + merge vs rebase (5 min)
- Monitoring stack — Prometheus, PromQL examples (5 min)
- Security — DevSecOps pipeline, RBAC, secrets (5 min)
- HA/DR — 99.9% SLA strategy, RTO/RPO (5 min)
- 3 STAR stories ready (conflict, failure, achievement) (5 min)

3-Hour Revision

- Everything in 1-hour + add:
- Read through Part 2 (Azure DevOps) completely (20 min)
- Read through Part 3 (Terraform) — focus on advanced + troubleshooting (20 min)
- Read through Part 5 (Kubernetes) — YAMLs + troubleshooting top 25 (20 min)
- Practice 20 scenario questions aloud (20 min)
- Review architecture questions — practice explaining trade-offs (20 min)
- Review Docker Compose, volumes, networking (10 min)
- Practice "What are your weaknesses?" and other HR questions (10 min)
- Review Azure Key Vault, ACR, AKS comparisons (10 min)
- Rest and hydrate (10 min)

6-Hour Revision (Full Preparation)

- Hour 1: Behavioral + Project walkthrough practice (speak aloud)
- Hour 2: Azure DevOps (Part 2) — full read, practice YAML from memory
- Hour 3: Terraform (Part 3) — full read, practice resource blocks from memory
- Hour 4: Kubernetes (Part 5) — YAMLs + troubleshooting scenarios
- Hour 5: Linux (Part 6) + Docker (Part 4) — command practice
- Hour 6: Architecture (Part 8) + Scenarios (Part 9) + HR + salary prep
- Last 15 min: Cheat sheets review + deep breaths

Mock Interview Questions

Easy Level (50 Questions)

#	Question
1	What is DevOps?
2	What is CI/CD?
3	What is Docker?
4	Difference between container and VM?
5	What is Kubernetes?
6	What is a Dockerfile?

#	Question
7	What is `git merge`?
8	What is Terraform?
9	What is Infrastructure as Code?
10	What is a Linux process?
11	What does `chmod 755` mean?
12	What is a load balancer?
13	What is DNS?
14	What is SSH?
15	What is YAML?
16	What is a microservice?
17	What is monitoring?
18	What is Prometheus?
19	What is Grafana?
20	What is an environment variable?
21	What is a Git branch?
22	What is a pull request?
23	What is a deployment pipeline?
24	What is a reverse proxy?
25	What is Nginx?
26	What is a cron job?
27	What is systemd?
28	What is TCP vs UDP?
29	What is a VPN?
30	What is RBAC?
31	What is a secret in Kubernetes?
32	What is a ConfigMap?
33	What is `docker compose up`?
34	What is a volume in Docker?
35	What is `terraform init`?
36	What is `terraform plan`?
37	What is a resource in Terraform?

#	Question
38	What is a Kubernetes Pod?
39	What is a Kubernetes Service?
40	What is a namespace?
41	What is `kubectl`?
42	What is a Jenkins pipeline?
43	What is Ansible?
44	What is a firewall?
45	What is HTTPS?
46	What is base64 encoding?
47	What is a health check?
48	What is high availability?
49	What is a backup?
50	What is an SLA?

Medium Level (50 Questions)

#	Question
1	ENTRYPOINT vs CMD in Docker?
2	How does Docker layer caching work?
3	Deployment vs StatefulSet in K8s?
4	How do you debug CrashLoopBackOff?
5	`git merge` vs `git rebase`?
6	Explain Terraform state and why it matters
7	What is a multi-stage Docker build?
8	How does Kubernetes service discovery work?
9	Explain Prometheus pull-based model
10	What is a Kubernetes Ingress?
11	How do you handle secrets in CI/CD?
12	What is a rolling update strategy?
13	Explain count vs for_each in Terraform
14	What are Kubernetes probes and their types?
15	How do you optimize Docker image size?

#	Question
16	What is `terraform import`?
17	How do you handle merge conflicts?
18	What is an HPA in Kubernetes?
19	Explain PromQL `rate()` function
20	What is a NetworkPolicy?
21	How does Jenkins credential store work?
22	What is a Kubernetes DaemonSet?
23	Explain Azure DevOps Service Connections
24	What is `terraform workspace`?
25	How do you implement blue-green deployment?
26	What is Docker Compose networking?
27	Explain Kubernetes RBAC
28	What is a Terraform module?
29	How do you set up alerting with Alertmanager?
30	What is PV and PVC in Kubernetes?
31	How do you handle log aggregation at scale?
32	What is a Kubernetes Job vs CronJob?
33	Explain branch protection rules
34	What is Azure Pipelines YAML vs Classic?
35	How do you handle database HA?
36	What is init container in K8s?
37	How do you secure SSH?
38	What is `terraform state rm`?
39	How does Kubernetes DNS work?
40	What is container vulnerability scanning?
41	Explain SLI vs SLO vs SLA
42	What is a sidecar pattern?
43	How do you manage Terraform state in a team?
44	What is Fluent Bit vs Fluentd?
45	How do you handle Kubernetes upgrades?
46	What is a Helm chart?

#	Question
47	Explain Docker networking modes
48	What is `terraform lifecycle`?
49	How do you implement canary deployments?
50	What is Kubernetes admission controller?

Hard Level (50 Questions)

#	Question
1	Design a CI/CD pipeline for a microservices architecture
2	Your Terraform state is corrupted — walk me through recovery
3	A pod keeps getting OOMKilled despite having 2GB limit — diagnose
4	Design a monitoring strategy for 100+ microservices
5	How would you implement zero-downtime database migration?
6	Explain how Patroni handles PostgreSQL failover
7	Design a multi-environment infrastructure with Terraform
8	Your Kubernetes cluster has DNS resolution failures — troubleshoot
9	How would you implement GitOps for 20+ applications?
10	Explain Terraform state locking internals
11	Design a disaster recovery plan with 30-minute RTO
12	Your Jenkins master is down — what's your recovery plan?
13	How do you handle secret rotation across 50+ services?
14	Design an auto-scaling strategy for unpredictable traffic
15	Your Docker build is 30 minutes — optimize it to 5 minutes
16	A Kubernetes node has disk pressure — walk through resolution
17	How do you implement compliance-as-code?
18	Design a logging pipeline that handles 1TB/day
19	Your deployment caused a 10% error rate increase — handle it
20	Explain AKS vs EKS vs self-managed Kubernetes trade-offs
21-50	*(Advanced networking, service mesh decisions, multi-cloud terraform, etcd backup/restore, K8s certificate renewal, custom Prometheus exporters, Grafana dashboard design, pipeline security hardening, cost optimization at scale, capacity planning methodology, chaos engineering implementation, platform engineering strategy, migration from monolith to microservices, container runtime comparison, Kubernetes operator pattern, advanced PromQL, Terraform provider development, incident response automation, SRE practices implementation, production readiness review)*

Expert Level (50 Questions)

#	Question
1	Design the entire infrastructure for a 500-developer organization
2	How would you build an Internal Developer Platform?
3	Architect a multi-region active-active deployment
4	Design a platform engineering team's operating model
5	How do you measure and improve DORA metrics at scale?
6	Design a Terraform module strategy for 100+ teams
7	Architect a zero-trust network for Kubernetes
8	How would you migrate 200 VMs to Kubernetes?
9	Design an observability platform supporting 10,000 containers
10	How do you implement progressive delivery at enterprise scale?
11-50	*(Enterprise RBAC strategy, multi-cluster management, FinOps at scale, compliance automation framework, developer experience measurement, SRE toil budget management, incident management maturity model, technical debt quantification, vendor evaluation framework, capacity planning for 10x growth, security audit automation, cloud cost governance, infrastructure testing strategy, change management at scale, release engineering practices, production readiness criteria, blameless postmortem culture, on-call health metrics, knowledge management system, technology radar process)*

Principal DevOps Level (50 Questions)

#	Question
1	How would you establish a DevOps Center of Excellence?
2	Design an organization-wide platform strategy
3	How do you drive cultural change for DevOps adoption?
4	Architect a global deployment pipeline for 30+ teams across 5 regions
5	How do you balance innovation with stability?
6	Design a technology governance framework
7	How do you evaluate build-vs-buy for infrastructure tooling?
8	Architect a self-service infrastructure platform
9	How do you handle a 100-service platform migration?
10	Design an SRE practice from scratch
11-50	*(C-level communication of technical strategy, budget management for infrastructure, vendor negotiation, staffing and team structure, cross-functional alignment, innovation labs, tech debt payoff strategy, engineering excellence metrics, developer productivity measurement, platform team OKRs, incident command system, regulated industry DevOps, acquisition integration, legacy modernization roadmap, edge*

#	Question
	computing strategy, AI/ML infrastructure, sustainability in cloud, open-source contribution strategy, conference speaking preparation, technical writing and documentation leadership)

> ***You've completed the full handbook. Want me to generate an even deeper version focused specifically on Azure DevOps, Terraform, Kubernetes, and consulting-style architecture questions?***